Priority Queue Study Guide

1. Introduction to Priority Queue

A Priority Queue is an abstract data type that operates similar to a regular queue, but with one important difference: each element has a priority associated with it. Elements are served based on their priority rather than their order of insertion.

Key Characteristics:

- Elements with higher priority are dequeued before elements with lower priority
- If two elements have the same priority, they are served according to their order in the queue
- It's an extension of the regular queue with priority handling

Priority Queue in Java

Java provides the `PriorityQueue` class in the `java.util` package. By default, it implements a min-heap where the smallest element has the highest priority.

```java
import java.util.PriorityQueue;

// Creating a Priority Queue
PriorityQueue<Integer> pq = new PriorityQueue<>();

// Adding elements
pq.add(10);
pq.add(5);
pq.add(15);
pq.add(2);

// Removing elements (smallest first)
System.out.println(pq.poll()); // Output: 2
System.out.println(pq.poll()); // Output: 5
```

Custom Priority with Comparator

You can define custom priority using a Comparator:

```java
// Max-heap (largest element has highest priority)
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());

// Custom comparator for objects
PriorityQueue<String> pqString = new PriorityQueue<>((a, b) -> b.length() - a.length());
```

Common Methods:

- `add(element)` or `offer(element)`: Insert an element
- `poll()`: Remove and return the highest priority element
- `peek()`: Return the highest priority element without removing
- `size()`: Get the number of elements
- `isEmpty()`: Check if the queue is empty

2. Usage and Applications

Real-World Applications

Task Scheduling in Operating Systems:

```java
class Task {
    String name;
    int priority;

    Task(String name, int priority) {
        this.name = name;
        this.priority = priority;
    }
}

PriorityQueue<Task> taskQueue = new PriorityQueue<>((a, b) -> b.priority - a.priority);
taskQueue.add(new Task("Critical Update", 10));
taskQueue.add(new Task("Background Backup", 3));
taskQueue.add(new Task("User Interface", 7));
```

Dijkstra's Algorithm for Shortest Path:

```java
class Node {
    int vertex;
    int distance;

    Node(int vertex, int distance) {
        this.vertex = vertex;
        this.distance = distance;
    }
}

PriorityQueue<Node> dijkstraQueue = new PriorityQueue<>((a, b) -> a.distance - b.distance);
```

Emergency Room Patient Management:

```java
class Patient {
    String name;
    int severity; // 1 = critical, 5 = minor

    Patient(String name, int severity) {
        this.name = name;
        this.severity = severity;
    }
}

PriorityQueue<Patient> emergencyQueue = new PriorityQueue<>((a, b) -> a.severity - b.severity);
```

Practical Example: Event Simulation

```java
import java.util.*;

class Event {
    int time;
    String description;

    Event(int time, String description) {
        this.time = time;
        this.description = description;
    }

    @Override
    public String toString() {
        return "Time " + time + ": " + description;
    }
}

public class EventSimulator {
    public static void main(String[] args) {
        PriorityQueue<Event> eventQueue = new PriorityQueue<>((a, b) -> a.time - b.time);

        eventQueue.add(new Event(5, "Meeting starts"));
        eventQueue.add(new Event(2, "Send email"));
        eventQueue.add(new Event(8, "Lunch break"));
        eventQueue.add(new Event(1, "Check calendar"));

        System.out.println("Events in chronological order:");
        while (!eventQueue.isEmpty()) {
            System.out.println(eventQueue.poll());
        }
    }
}
```

Performance Benefits:

- **Insertion**: O(log n)
- **Deletion**: O(log n)
- **Peek**: O(1)
- **Space Complexity**: O(n)

3. Heap Sort

Heap Sort is a comparison-based sorting algorithm that uses a binary heap data structure. It's closely related to Priority Queue since Priority Queue is typically implemented using a heap.

How Heap Sort Works:

1. **Build a Max Heap** from the input array
2. **Extract the maximum** element (root) and place it at the end
3. **Heapify** the remaining elements
4. **Repeat** until all elements are sorted

Implementation Using Priority Queue:

```java
import java.util.*;

public class HeapSort {

    public static void heapSort(int[] arr) {
        // Use max-heap for ascending order
        PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());

        // Add all elements to the heap
        for (int num : arr) {
            maxHeap.add(num);
        }

        // Extract elements in descending order and place at the end
        for (int i = arr.length - 1; i >= 0; i--) {
            arr[i] = maxHeap.poll();
        }
    }

    public static void main(String[] args) {
        int[] arr = {64, 34, 25, 12, 22, 11, 90};

        System.out.println("Original array: " + Arrays.toString(arr));
        heapSort(arr);
        System.out.println("Sorted array: " + Arrays.toString(arr));
    }
}
```

Manual Heap Sort Implementation:

java

```java
public class ManualHeapSort {

    public void sort(int arr[]) {
        int n = arr.length;

        // Build max heap
        for (int i = n / 2 - 1; i >= 0; i--) {
            heapify(arr, n, i);
        }

        // Extract elements from heap one by one
        for (int i = n - 1; i > 0; i--) {
            // Move current root to end
            int temp = arr[0];
            arr[0] = arr[i];
            arr[i] = temp;

            // Call heapify on the reduced heap
            heapify(arr, i, 0);
        }
    }

    void heapify(int arr[], int n, int i) {
        int largest = i; // Initialize largest as root
        int left = 2 * i + 1;
        int right = 2 * i + 2;

        // If left child is larger than root
        if (left < n && arr[left] > arr[largest]) {
            largest = left;
        }

        // If right child is larger than largest so far
        if (right < n && arr[right] > arr[largest]) {
            largest = right;
        }

        // If largest is not root
        if (largest != i) {
            int swap = arr[i];
            arr[i] = arr[largest];
            arr[largest] = swap;
```

```
            // Recursively heapify the affected sub-tree
            heapify(arr, n, largest);
        }
    }
}
```

Heap Sort Characteristics:

- **Time Complexity**: O(n log n) in all cases
- **Space Complexity**: O(1) - in-place sorting
- **Stability**: Not stable (relative order of equal elements may change)
- **Adaptive**: No, always takes O(n log n) time

Quiz Questions

1. What is the default behavior of Java's PriorityQueue? a) Max-heap (largest element has highest priority) b) Min-heap (smallest element has highest priority) c) FIFO (First In, First Out) d) LIFO (Last In, First Out)

Answer: b) Min-heap (smallest element has highest priority)

2. What is the time complexity of inserting an element into a Priority Queue? a) O(1) b) O(log n) c) O(n) d) O(n log n)

Answer: b) O(log n)

3. Which method removes and returns the highest priority element from a PriorityQueue? a) remove() b) peek() c) poll() d) pop()

Answer: c) poll()

4. What is the worst-case time complexity of Heap Sort? a) O(n) b) O(n log n) c) $O(n^2)$ d) O(log n)

Answer: b) O(n log n)

5. In a max-heap used for heap sort, where is the largest element located? a) At any leaf node b) At the last position c) At the root (index 0) d) At the middle of the array

Answer: c) At the root (index 0)

Practice Coding Questions

1. Hospital Emergency System Create a Priority Queue system for a hospital emergency room. Implement a Patient class with name and severity level (1-10, where 1 is most critical). Write a program that processes patients in order of severity, and if two patients have the same severity, process them in the order they arrived.

2. Task Scheduler Design a task scheduling system using Priority Queue. Each task has a name, priority level, and estimated duration. Write methods to:

- Add a new task
- Execute the highest priority task
- Display all pending tasks in priority order
- Calculate total time for all pending tasks

3. Merge K Sorted Arrays Given K sorted arrays, use a Priority Queue to merge them into one sorted array. Each array element should be wrapped in a class that tracks which array it came from and its index within that array.

4. Custom Heap Sort Implement a heap sort algorithm that can sort in both ascending and descending order based on a parameter. Your implementation should work with generic types and accept a custom Comparator.

5. Event Management System Create an event management system where events have a scheduled time and description. Use Priority Queue to:

- Schedule new events
- Get the next upcoming event
- Process all events for a given day in chronological order
- Handle event conflicts (events at the same time)